

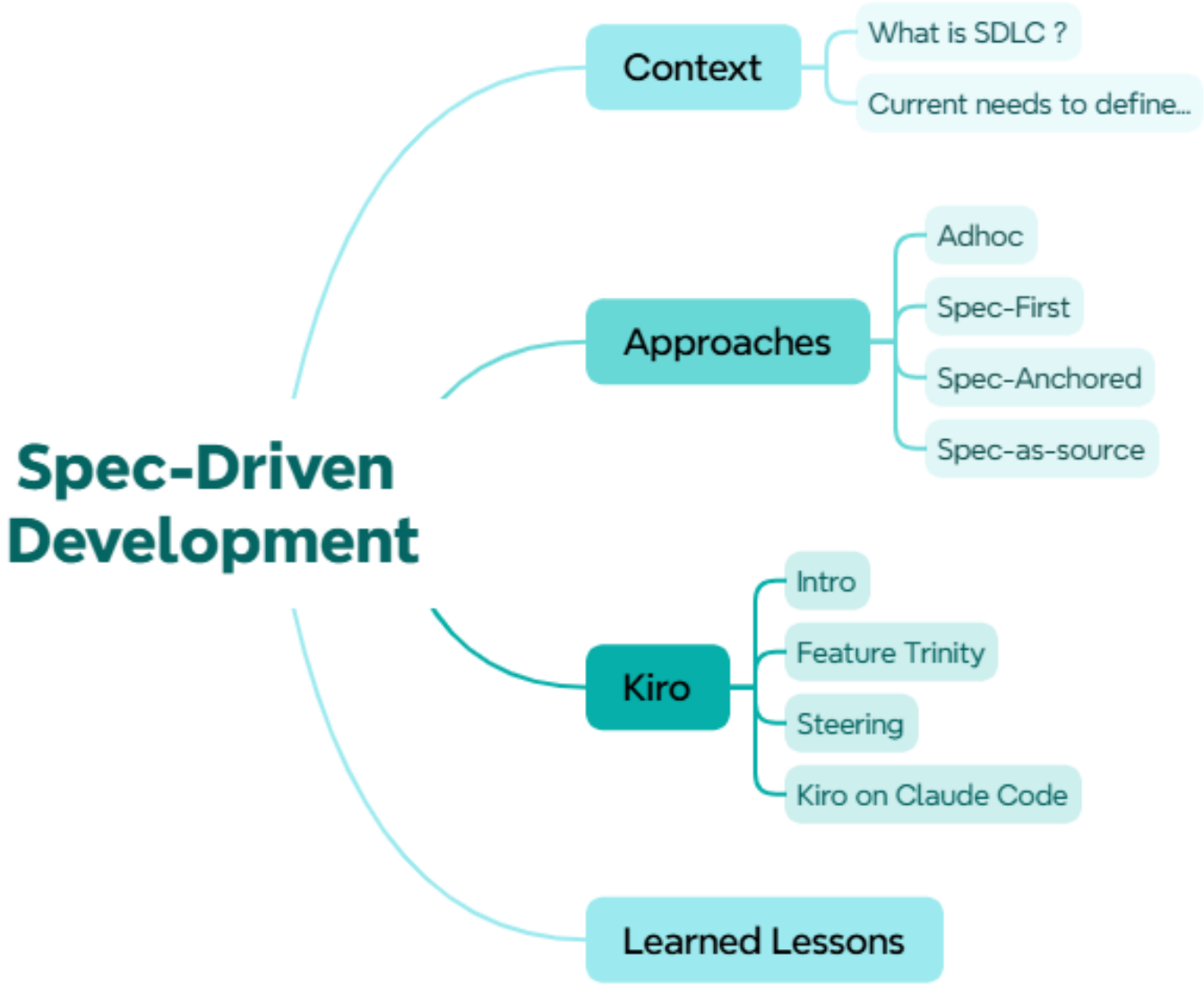
Spec-Driven Development

Arquitectura de Software II

Universidad de La Sabana
FACULTAD DE INGENIERÍA

Universidad de La Sabana
FACULTAD DE INGENIERIA

Agenda



My interest is to choose a simple framework to reduce number of iterations (time and token consumption).
Out-of-Scope: Agentic Workflow: SPD+SDD+Harness

Universidad de La Sabana
FACULTAD DE INGENI

What is SDLC ?



Vibe Coding

Reactive, prompt-heavy and fragile

SDD

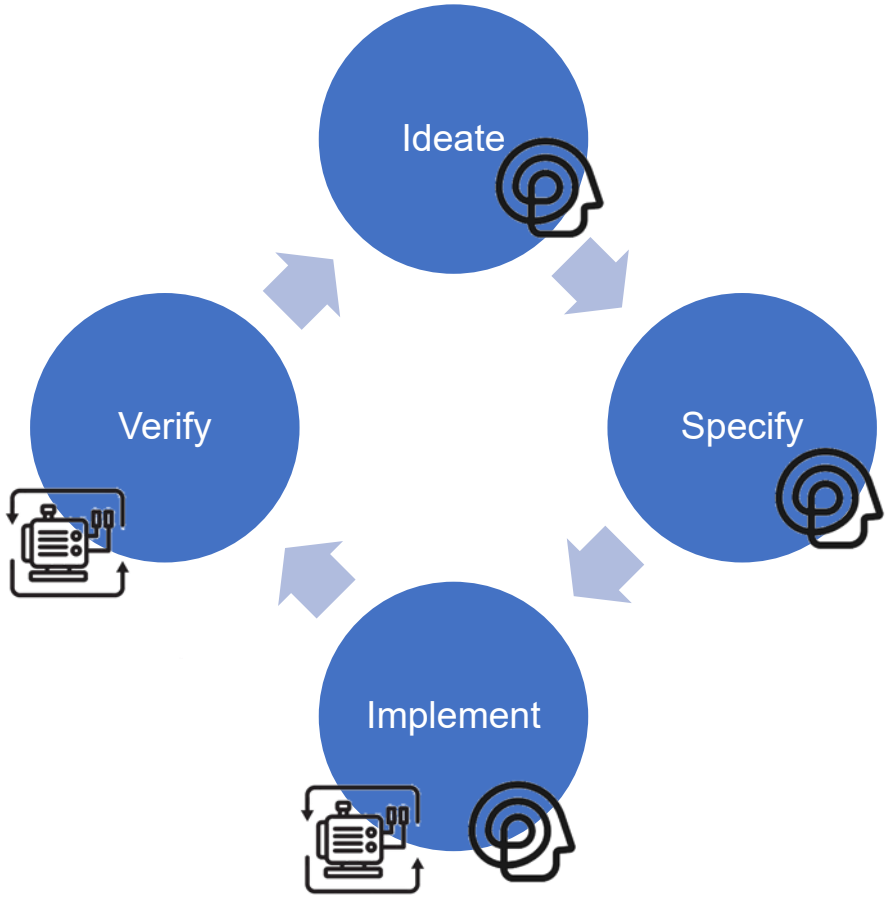
Documented and verifiable

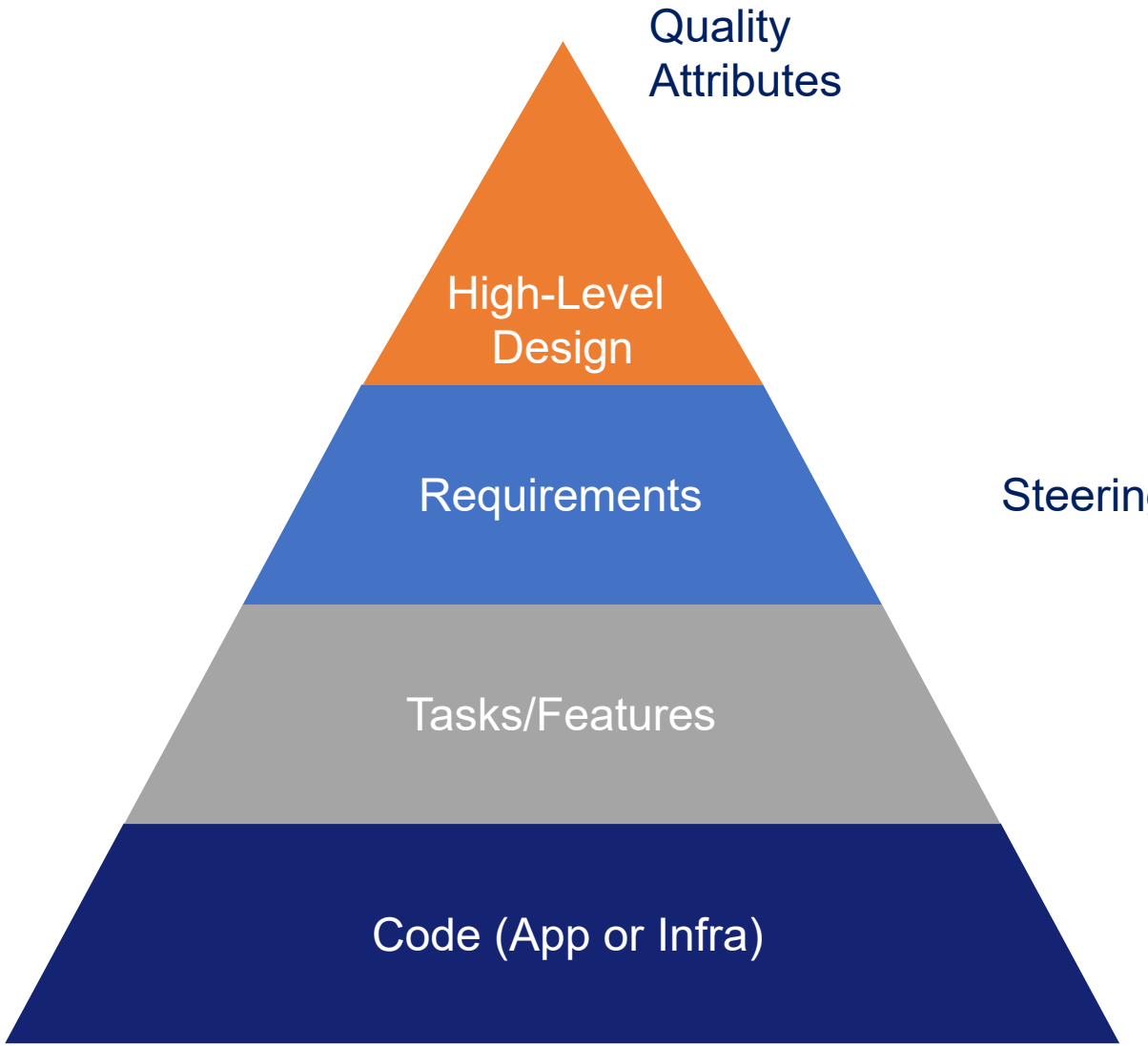
Feature	Waterfall	Agile	Vibe Coding (Chat AI)	Spec-Drive Development (SDD)
Role of Doc	Rigid Contract	Minimal Guidance	N/A	Executable Blueprint (Living Doc)
Context Management	Contract	Human Memory	Token Window	Local Index & Spec Files
Source of Truth	Front Doc	Working Software		Living Specification
Repeatability	Components/Libraries	Manual	Hit or Miss	Deterministic Plans
Code Quality	Human PR Review	Human PR Review	Review?	Spec Validated

Universidad de La Sabana
FACULTAD DE INGENIERIA

Needs to define...

Objective: Define an organized method to manage the migration/reorg of workloads.





Universidad de La Sabana
FACULTAD DE INGENIERIA

Approaches for SDD

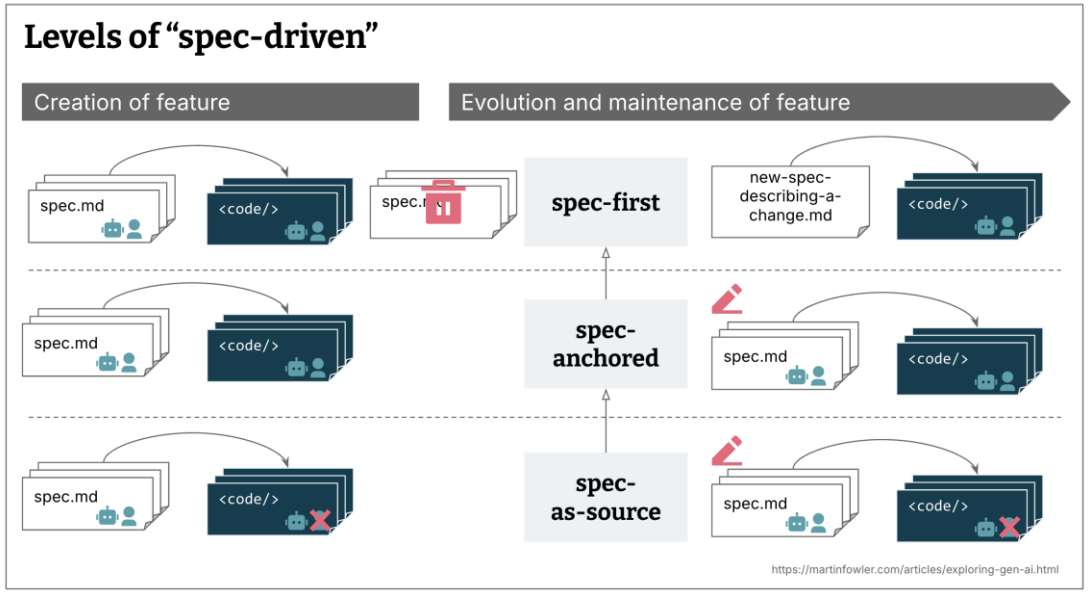
Evolution of BDD and TDD

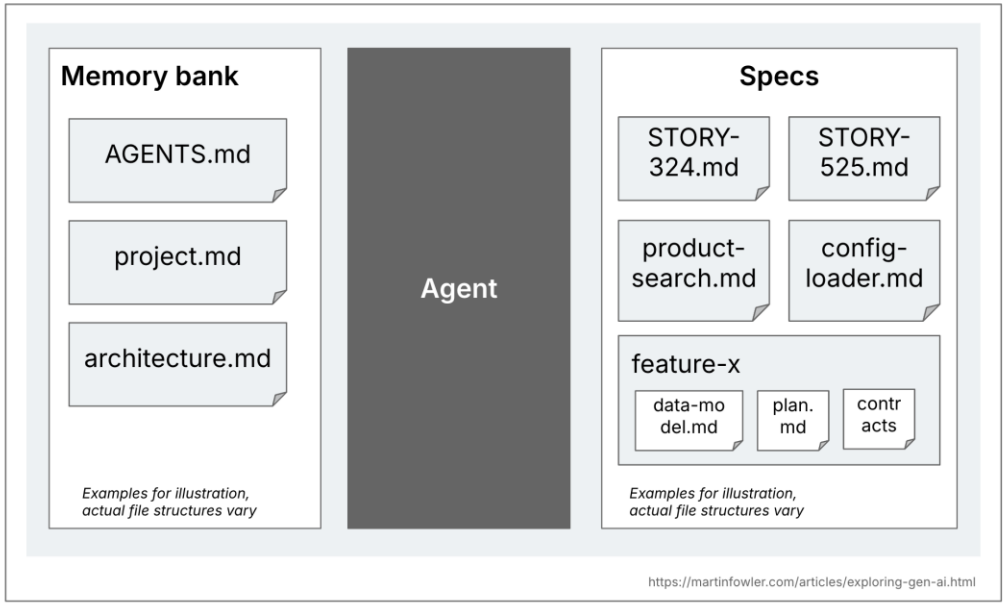
3 approaches:

- Spec-First
- Spec-Anchored
- Spec-as-source.

Differences due to modifications on initial specifications and code.
Structured similar: Memory bank, specs, features/tasks

There are another SDD frameworks, well-mature such as GitHub SpecKit, or OpenSpec; or methodologies such as BMAD. For simplicity, I use Kiro.

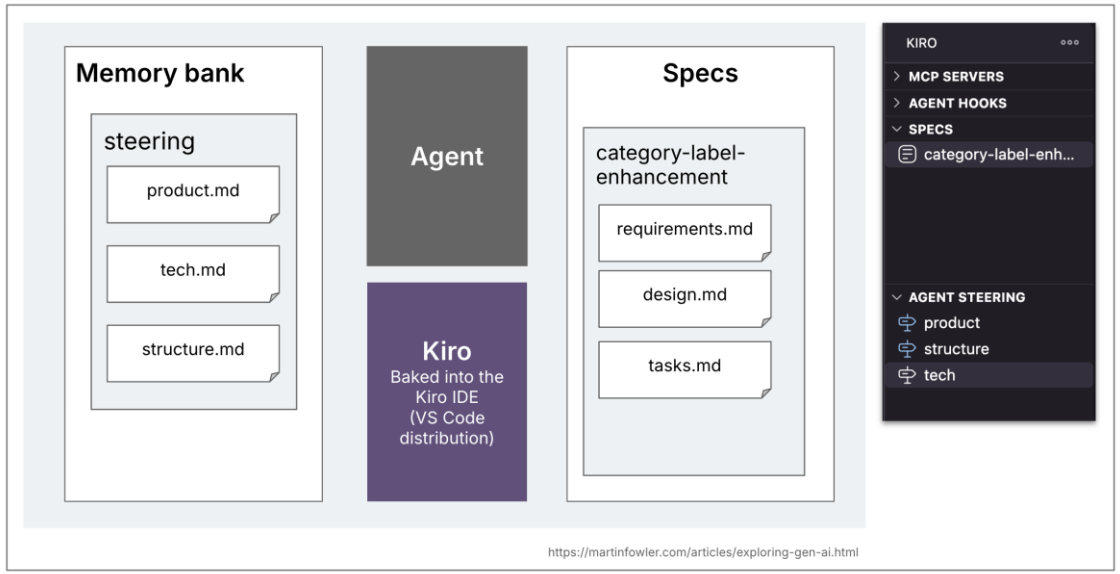


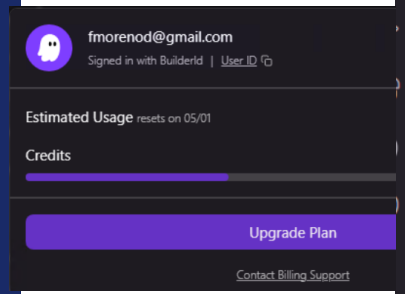


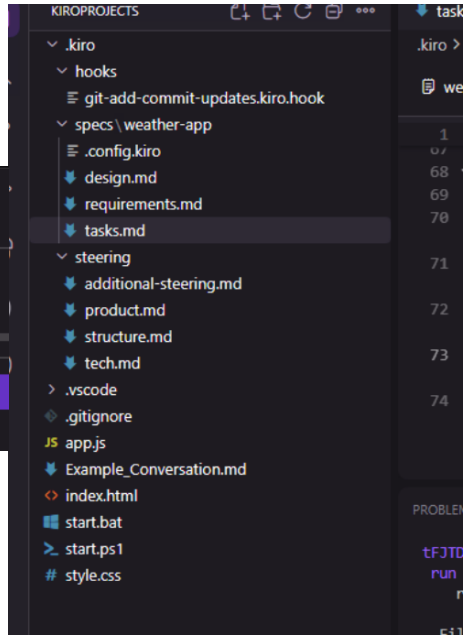
Universidad de La Sabana
FACULTAD DE INGENIERIA

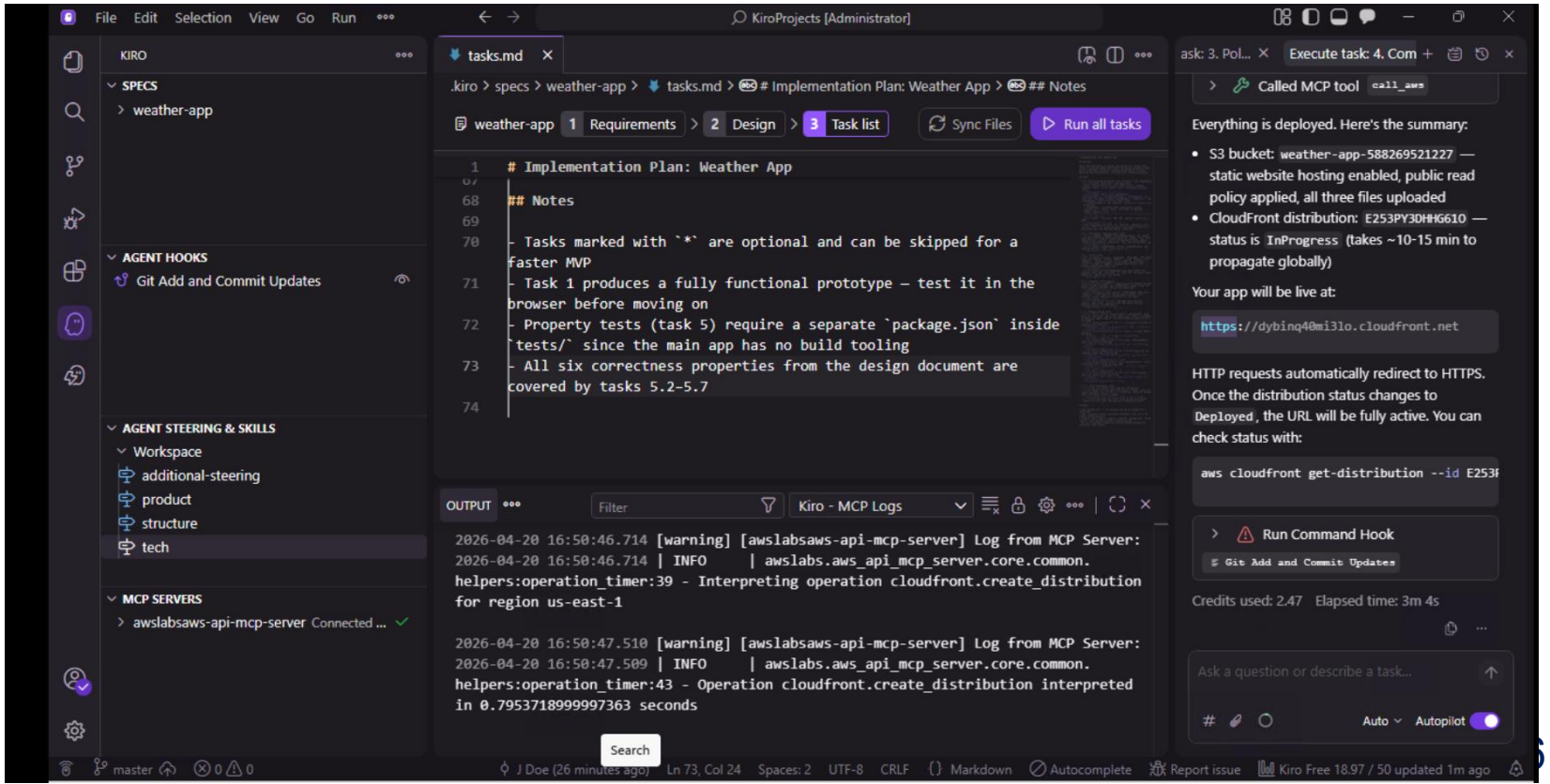
Kiro

- AI IDE with focus on Spec-Driven Dev, for features but support bug fixing.
- AWS managed and charge LLM.
- AWS Support and Training
- Simple and intuitive to learn



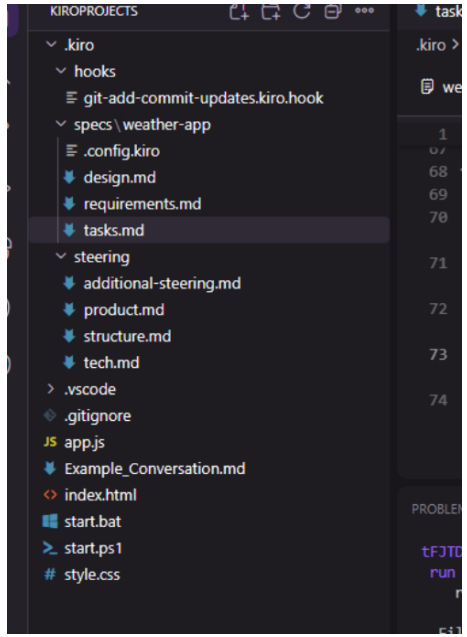






Universidad de La Sabana
FACULTAD DE INGENIERIA

Kiro: Feature Trinity/Three-Phased Workflow



Initial Prompt

Requirements

- Functional Requirements. Captures user intentions and its constraints. Defined what needs to do it in EARS syntax.

Design

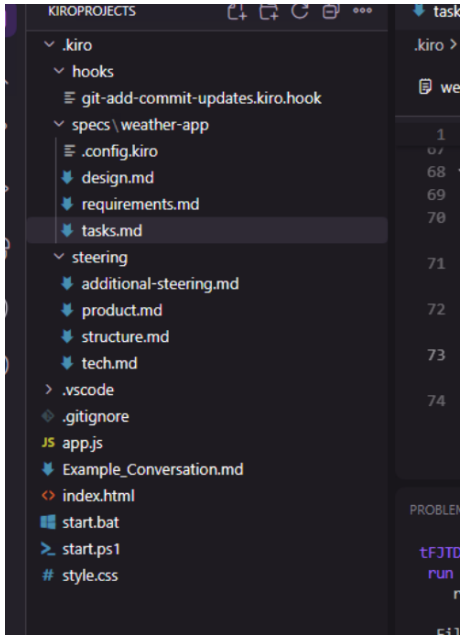
- Schema abouts how to do it. State machines and relationships.

Tasks

- Executable plan. Atomic sequence plan that agent has to follow.

Universidad de La Sabana
FACULTAD DE INGENIERIA

Kiro: Steering



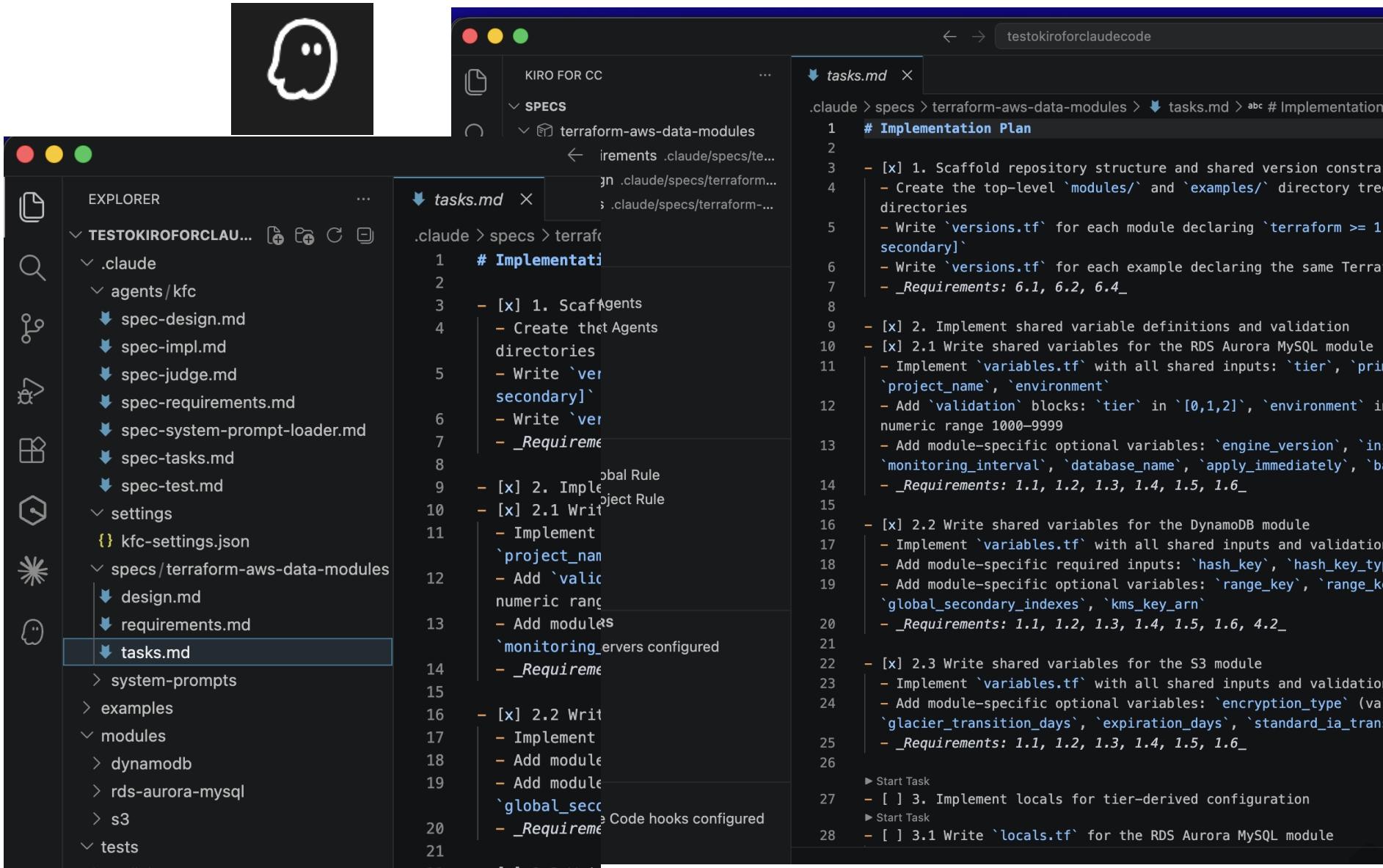
Define general rules that aren't link to the feature directly. There are examples for government and tech definitions, such as:

- Tech Stack: Frontend with React and Tailwind CSS. Backend with Python 3, FastAPI. DB with PostgreSQL and Prisma as ORM
- Patterns: Using Domain-Driven Development. All code has to have a header documentation about purpose of the file and methods.
- Naming definition: Use PascalCase for components names and camelCase for file names.

Universidad de La Sabana
FACULTAD DE INGENIERIA

Kiro on Claude Code

- VSCode Extension
- Need a Claude Code API Keys
- Bypass Security to act as agent
- Create more under-the-hood files (.claude/agents)



Universidad de La Sabana
FACULTAD DE INGENIERIA

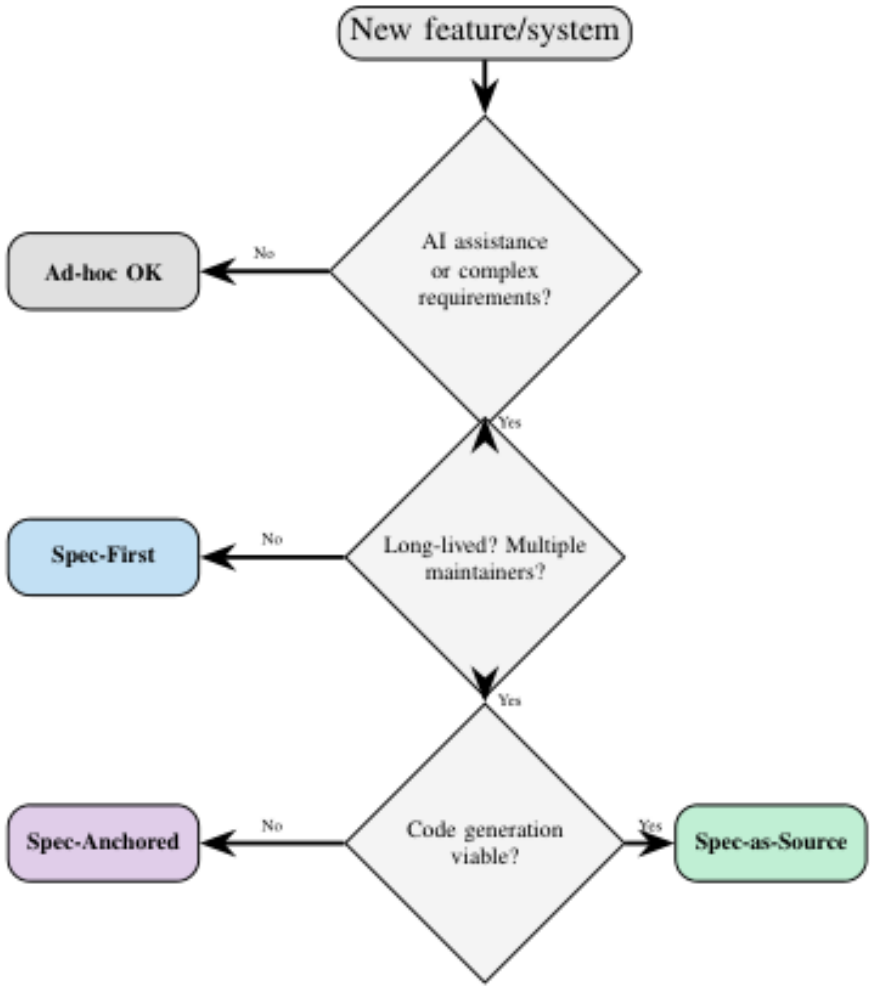
Out of Scope

My recommendations are:

- **Additional loops:** AI Harnessing (env), Context Eng(-as-a-Code; the product is the spec), Eval-Driven Design.
- **Branching strategy:** Use gitflow for complex projects, otherwise use trunk-based strategy for simplicity.
- **Context Window:** Depends on scale (member number) and distribution of the project, it can be a problem. Try to reduce limit information on the trinity or use limited information, i.e. LLM Wiki and OpenViking
- **Deployment:** Use several type of testing on early steps -focus on integration-, to avoid rework on CI/CD pipelines.
- **Human on the loop:** Consequences of avoiding this step: Reputational damage, dismissal, legal issues, bankrupt, or layoffs.

Universidad de La Sabana
FACULTAD DE INGENIERIA

Learned Lessons



- Follow a method to scale on a team and choose the right approach based on team' maturity and maintainability.
- Need of an orchestrator (stages and transversal policies) between projects (Pipeline).
- Difference between steering and structured files (design, requirements, tasks)
- Allow optional tasks for testing (Validation tasks are longer) or using external tools.
- Modifications has to be saved on documents or on steering to have single source of truth, otherwise you can save interactions on Export Conversations.
- Cost is based on the complexity and tier.

Universidad de La Sabana
FACULTAD DE INGENIERIA

References

About Spec-Driven Dev:
arXiV (Cornell) ,
MartinFowler and
DeepLearning

About Kiro:
SkillBuilder Courses [1, 2]

About Claude Code
Anthropic Courses [A, B]



Spec-Driven Development:
From Code to Contract in the Age of AI Coding Assistants

Deepak Babu Piskala
Seattle, USA
Technical Report

Abstract—The rise of AI coding assistants has reignited interest in an old idea: what if specifications—not code—were the primary artifact of software development? Spec-driven development (SDD) inverts the traditional workflow by treating specifications as the source of truth and code as a generated or verified secondary artifact. This paper provides practitioners with a comprehensive guide to SDD, covering its principles,

martinFowler.com
Refactoring Agile Architecture About Thoughtworks

Understanding Spec-Driven-Development:
Kiro, spec-kit, and Tessl



This article is part of "Exploring Gen AI". A series capturing Thoughtworks technologists' explorations of using gen ai technology for software development.

15 October 2025

Birgitta Böckeler



Birgitta is a Distinguished Engineer and AI-assisted delivery expert at Thoughtworks. She has over 20 years of experience as a software developer, architect and technical leader.

aws training and certification

Structured Approach to AI coding with Spec-Driven Development on Kiro

Lab Information
Not started
1 hour 30 minutes
Available languages
Feedback
What's New

AWS Learning Assistant is here!
Use our AI-powered learning assistant to guide you through AWS co
code as you work through the lab.

Spec-Driven Development with Kiro

ANTHROP\C

Building with the Claude API

Universidad de La Sabana
FACULTAD DE INGENIERIA

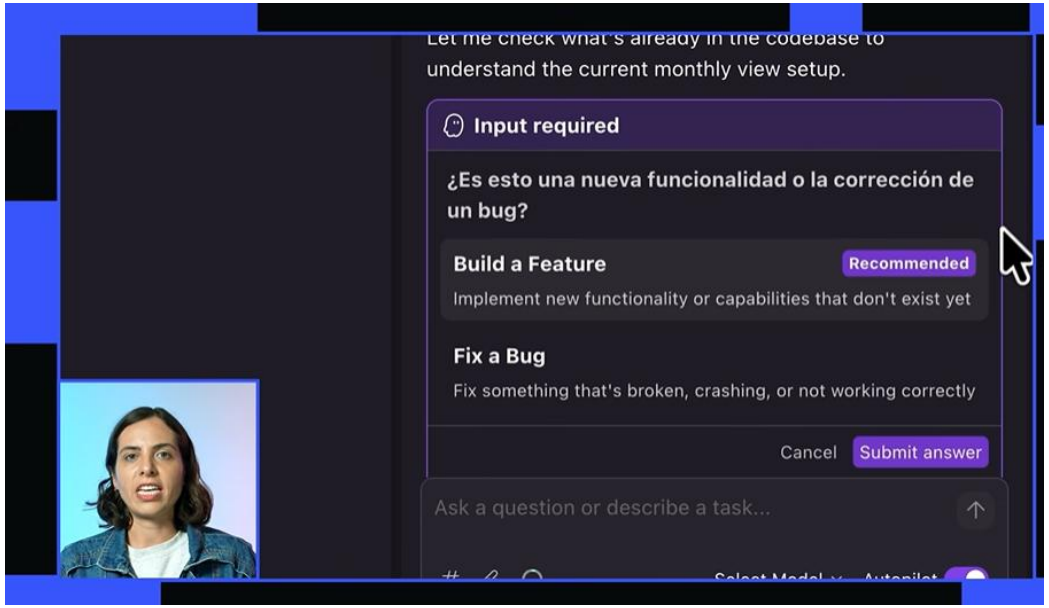
Demo

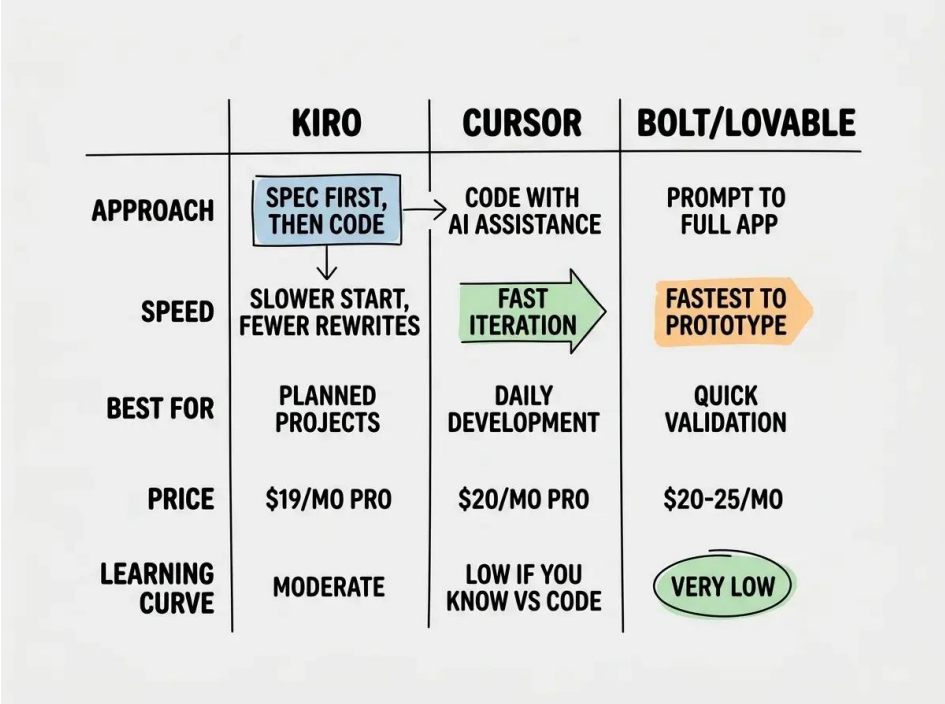
Actúa como Serverless Solutions Architect especializado en AWS CloudFormation y AWS SAM (Serverless Application Model).

Necesitamos diseñar la infraestructura para una API Serverless de alta disponibilidad para la gestión de productos.

Requerimientos clave:
*- **Database**: Tabla de Amazon DynamoDB con `BillingMode: PAY_PER_REQUEST` (On-Demand), clave primaria `productId` (String) y un Global Secondary Index (GSI) sobre `category` (String).*
*- **Compute**: Función AWS Lambda en Python 3.12 / Node.js 20 con arquitectura ARM64 (`graviton2`), tiempo de ejecución límite (timeout) de 10 segundos y permisos mínimos IAM usando políticas administradas de SAM (`DynamoDBCrudPolicy`).*
*- **API Layer**: API Gateway HTTP API (`AWS::Serverless::HttpApi`) integrado con la función Lambda mediante ruta `POST /products`.*
*- **Observability**: Grupo de logs en CloudWatch (`AWS::Logs::LogGroup`) con retención configurable de 14 días y rastreo activo con AWS X-Ray habilitado.*

Requerimientos para el IaC:
- Parámetros configurables: `EnvironmentName` (dev/prod), `LogLevel` (INFO/DEBUG).
- Sección `Globals` para definir configuraciones comunes de Lambdas (timeout, runtime, tracing).
- Sección `Outputs` exportando el endpoint de la API (`ApiUrl`) y el ARN de la tabla DynamoDB.





Universidad de La Sabana
FACULTAD DE INGENIERIA

EARS Quick Reference

Aalto University

EARS quick reference sheet

Easy Approach to Requirements Syntax

Sentence types

Ubiquitous
- The <system name> shall <system response>
- *The kitchen system shall have an input hatch.*

Event-driven
- **When** <optional preconditions> <trigger>, the <system> shall <system response>
- *When the chef inserts a potato to the input hatch, the kitchen system shall peel the potato.*

State-driven
- **While** <in a state>, the <system> shall <system response>
- *While the kitchen system is in maintenance mode, the kitchen system shall reject all input.*

Unwanted behavior
- **If** *<optional preconditions> <trigger>,* ***then*** the <system> shall <system response>
- *If a spoon is inserted to the input hatch, then the kitchen system shall eject the spoon.*

Optional
- **Where** <feature>, the <system> shall <system response>
- *Where the kitchen system has a food freshness sensor, the kitchen system shall detect rotten foodstuffs.*

Steps to take in applying *EARS*

1. **Identify** whether you are working with a requirement, or something else (e.g. note or example)
2. Identify **compound requirements**, i.e. whether the requirement needs to be **split**
3. Identify the **acting system, person or process**
4. Analyse the needed **sentence type(s)**
5. Identify possible **missing requirements**
   - E.g. 2 states and 2 events usually produce 4 requirements
6. **Analyse** the translated requirements for **ambiguity, conflict and repetition**
7. **Review** requirements if possible
8. Iterate as required

Some characteristics of a **good requirement**

- **Unambiguous**
  - One interpretation
- **Traceable**
  - Has unique identifier
- **Consistent**
  - Doesn't conflict other requirements
- **Verifiable**
  - Possible to check system meets requirement
- **Complete**
  - Not lacking relevant information

Universidad de La Sabana
FACULTAD DE INGENIERIA

Examples

EARS Patterns for Requirements

Name	Pattern
Ubiquitous	**The <system name> shall <system response>**
Event-Driven	**WHEN** <trigger> <optional precondition> **the <system name> shall <system response>**
Unwanted Behavior	**IF** <unwanted condition or event>, **THEN** **the <system name> shall <system response>**
State-Driven	**WHILE** <system state>, **the <system name> shall <system response>**
Optional Feature	**WHERE** <feature is included>, **the <system name> shall <system response>**
Complex	(combinations of the above patterns)

/in/yacinetazerout

EARS Pattern Type	EARS Syntax / Keyword	EARS Requirements Example	When to Use It
Ubiquitous Always active	`The <system> shall <response>`	• The ATM shall encrypt all PIN entries. • The dosing pump shall log every infusion event to non-volatile memory.	Use it when there is no trigger and no condition — the requirement is always active.
State-driven During a state	`While <state>, the <system> shall <response>`	• While the vehicle is in reverse, the system shall display the rear camera feed. • While the device battery level is below 20%, the device shall reduce the screen brightness.	When the requirement stays active for as long as the condition lasts.
Event-driven After a trigger	`When <trigger>, the <system> shall <response>`	• When the customer submits an order, the order system shall generate an invoice. • When a work item moves to Approved, the system shall lock the requirement	When behavior happens after an event occurs. The trigger is a moment, not a stretch of time.
Unwanted behavior Fault handling	`If <trigger>, then the <system> shall <response>`	• If an uploaded file size exceeds 25 MB, then the system shall reject it and display a size-limit message. • If the sensor returns no reading for three cycles, then the controller shall fall back to the modeled value.	For faulty conditions, invalid inputs, timeouts, or misuse — anything you don't want to happen but that will. The pattern teams skip, and where the expensive gaps live.
Optional feature Variant-specific	`Where <feature is present>, the <system> shall <response>`	• Where the device includes a fingerprint sensor, the app shall offer biometric sign-in.	When products ship in variants with optional features.
Extended EARS Combined conditions	`While <state>, when <trigger>, the <system> shall <response>`	• While the account is locked, when a sign-in is attempted, the system shall log the attempt and notify the account owner.	When several conditions exist together.

Universidad de La Sabana

FACULTAD DE INGENIERÍA

CONTACTO

Francisco Moreno

Profesor Catedra – Facultad de Ingeniería

franciscomodi@unisabana.edu.co